

VERBUME

Gra do nauki słownictwa angielskiego

Dokumentacja projektowa · wersja 0.5.0

Aplikacja **Verbume** — Android, tryb **CLASSIC**

Wersja **0.5.0** (versionCode **5**)

Identyfikator **com.verbume.app**

Repozytorium **github.com/kubakozub/Verbume** — gałąź **master**, commit **55b508e**

Demo **kubakozub.github.io/Verbume**

Zawartość **102** zestawy · **2084** hasła · **14** działów matury podstawowej

Dokument **29** lipca 2026

Spis treści

1. Czym jest Verbume	2
1.1. Pomysł	2
1.2. Co odróżnia Verbume od fiszek	2
1.3. Dwie postaci aplikacji	2
2. Instalacja i uruchomienie	3
2.1. Telefon z Androidem	3
2.2. Przeglądarka	3
2.3. Budowanie ze źródeł	3
3. Instrukcja użytkownika	4
3.1. Ekran startowy i menu	4
3.2. Przebieg rozgrywki	4
3.3. Punkty, serie i poziomy	5
3.4. Pancierz i skrzynie z amunicją	6
3.5. Zestawy — własne i wbudowane	6
3.6. Opcje	7
3.7. Osiągnięcia i wyniki	7
4. Zawartość merytoryczna — słownictwo	8
4.1. Zakres i podział	8
4.2. Zasada dwóch słów	8
4.3. Dopasowanie odpowiedzi	8
4.4. Wielkość zestawu	9
5. Architektura	10
5.1. Stos technologiczny	10
5.2. Podział na moduły	10
5.3. Silnik gry	11
5.4. Zapis danych	11
5.5. Grafika i font bitmapowy	11
5.6. Demo przeglądarkowe	12
6. Testy	13
6.1. Co jest sprawdzane	13
6.2. Testowalność silnika	13
7. Decyzje projektowe	14
8. Realizacja dokumentu projektowego	15
9. Ograniczenia i kierunki rozwoju	16
A. Załącznik: struktura repozytorium	17

1. Czym jest Verbume

1.1. Pomysł

Verbume to gra na Androida do nauki angielskiego słownictwa. Nazwa składa się z łacińskiego *verbum* — słowo — i angielskiego *game*. Aplikacja powstała wprost z dokumentu projektowego *Verbume* — *Verbum* + *Game* i realizuje opisany tam tryb CLASSIC wraz z pełną progresją trudności.

Zasada jest prosta. Na środku planszy stoi działko. Z obu boków nadchodzą potworki, każdy z przypisanym angielskim słowem wypisanym nad głową. Gracz wpisuje polskie tłumaczenie; kiedy trafi, działko obraca się w stronę tego potworka i strzela. Kiedy potworek dojdzie do działka, gracz traci życie. Po utracie wszystkich żyć gra się kończy.

Mechanika wymusza coś, czego zwykle fiszki nie wymuszają: **aktywne odtworzenie słowa z pamięci pod presją czasu**. Nie ma listy odpowiedzi do wyboru ani przycisku „wiedziałem”. Albo gracz potrafi napisać tłumaczenie, zanim potworek dojdzie, albo nie.

1.2. Co odróżnia Verbume od fiszek

Element	Fiszki (np. Quizlet)	Verbume
Sprawdzenie wiedzy	gracz sam ocenia, czy wiedział	trzeba wpisać słowo — nie da się oszukać
Presja czasu	brak	potworek idzie w stronę działka
Wiele haseł naraz	jedno na raz	do siedmiu równocześnie, gracz wybiera kolejność
Motywacja do powtórek	poczucie obowiązku	punkty, mnożniki serii, poziomy, osiągnięcia
Częstotliwość kontaktu ze słowem	raz na przebieg	kilka razy w jednej rozgrywce

Ostatni wiersz jest powodem, dla którego zestawy mają po około dwadzieścia haseł, a nie po sto. Przy dwudziestu hasłach jedna rozgrywka pokazuje każde słowo kilkakrotnie i powtórka dzieje się sama. Przy stu hasłach słowo wraca raz na kwadrans, czyli praktycznie nigdy.

1.3. Dwie postaci aplikacji

Ta sama gra istnieje w dwóch wydaniach, zbudowanych z tego samego kodu silnika:

- **Aplikacja na Androida** — plik APK, działa bez internetu, zapisuje postępy na telefonie.
- **Demo przeglądarkowe** — jeden plik HTML odtwarzający tę samą mechanikę w JavaScriptcie. Służy do pokazania gry komuś, kto nie chce nic instalować: wystarczy otworzyć adres.

Istnieje też siostrzana aplikacja **Aleima** — ta sama gra, interfejs angielski, nauka hiszpańskiego, słownictwo podzielone według poziomów CEFR. Opisuje ją osobny dokument.

2. Instalacja i uruchomienie

2.1. Telefon z Androidem

Aplikacja nie jest publikowana w Google Play, więc instaluje się ją z pliku APK (`VerbumeV050.apk` , około 8,9 MB). Wymagany jest Android 7.0 lub nowszy.

1. Przenieś plik APK na telefon — przez kabel, chmurę albo pobierając go bezpośrednio w przeglądarce telefonu. Gmail odrzuca załączniki `.apk` , również spakowane w archiwum, więc poczta nie jest tu drogą.
2. Otwórz plik w menedżerze plików.
3. Android zapyta o zgodę na „instalowanie nieznanych aplikacji” — zgodę nadaje się temu programowi, z którego plik jest otwierany (menedżerowi plików albo przeglądarce).
4. Potwierdź instalację.

Kolejne wersje instalują się na wierzch poprzednich bez utraty danych, pod warunkiem że mają wyższy `versionCode` . Postępy gracza leżą w prywatnym katalogu aplikacji i znikają dopiero przy jej odinstalowaniu.

UWAGA O PODPISIE

Wydania są podpisane kluczem debugowym. Do nauki i pokazu to wystarcza, ale taki plik nie nadaje się do publikacji w sklepie i nie zaktualizuje aplikacji podpisanej innym kluczem.

2.2. Przeglądarka

Demo działa pod adresem kubakozub.github.io/Verbume i nie wymaga instalowania niczego. Cała gra to jeden plik HTML (188 KB) z grafiką wbudowaną w treść, więc działa też po pobraniu na dysk i otwarciu z pliku — również bez internetu.

Na komputerze gra się klawiaturą fizyczną, na telefonie klawiaturą narysowaną na ekranie. Postępy i własne zestawy zapisują się w pamięci przeglądarki (`localStorage`), osobno dla każdej przeglądarki i urządzenia.

2.3. Budowanie ze źródeł

Do zbudowania potrzebne są JDK 21 i Android SDK z platformą 35. Ścieżkę do SDK podaje się w pliku `local.properties` , którego celowo nie ma w repozytorium, bo jest zależny od maszyny:

```
echo "sdk.dir=/sciezka/do/Android/Sdk" > local.properties
./gradlew :app:assembleDebug          # APK w app/build/outputs/apk/debug/
./gradlew :app:testDebugUnitTest      # 44 testy jednostkowe
```

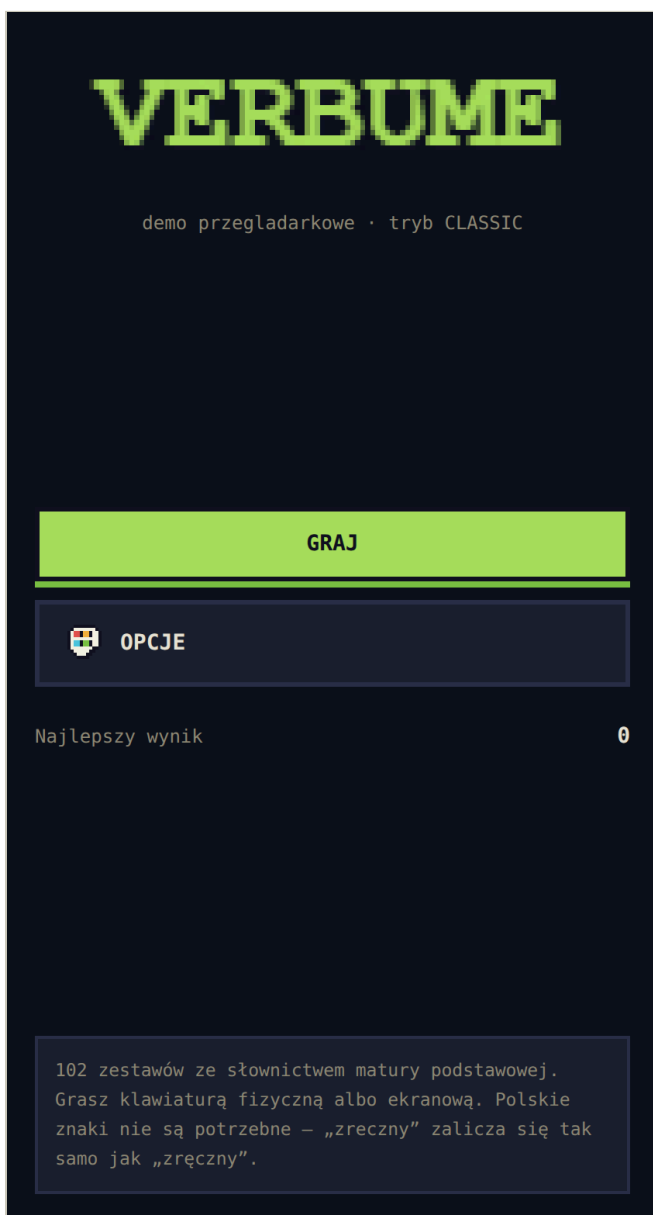
Gradle pobierze przy pierwszym uruchomieniu resztę zależności. Testy jednostkowe nie wymagają emulatora ani podłączonego telefonu — powód opisuje rozdział 6.

3. Instrukcja użytkownika

3.1. Ekran startowy i menu

Po uruchomieniu aplikacja proponuje grę anonimową. Dokument projektowy przewidywał również konta i logowanie, ale to wymaga serwera, którego ta wersja nie ma — dlatego wszystko zapisuje się lokalnie, a jedyną tożsamością gracza jest wpisana nazwa.

Menu główne prowadzi do sześciu miejsc: **GRAJ**, **ZESTAWY**, **OPCJE**, **OSIĄGNIĘCIA**, **WYNIKI** i **WYJŚCIE**.



Rys. 1. Menu główne. Cały interfejs, poza dłuższą prozą, rysowany jest fontem bitmapowym z arkusza grafiki — także polskie znaki.



Rys. 2. Wybór zestawu. Zestawy zgrupowane są według czternastu działów tematycznych matury podstawowej.

3.2. Przebieg rozgrywki

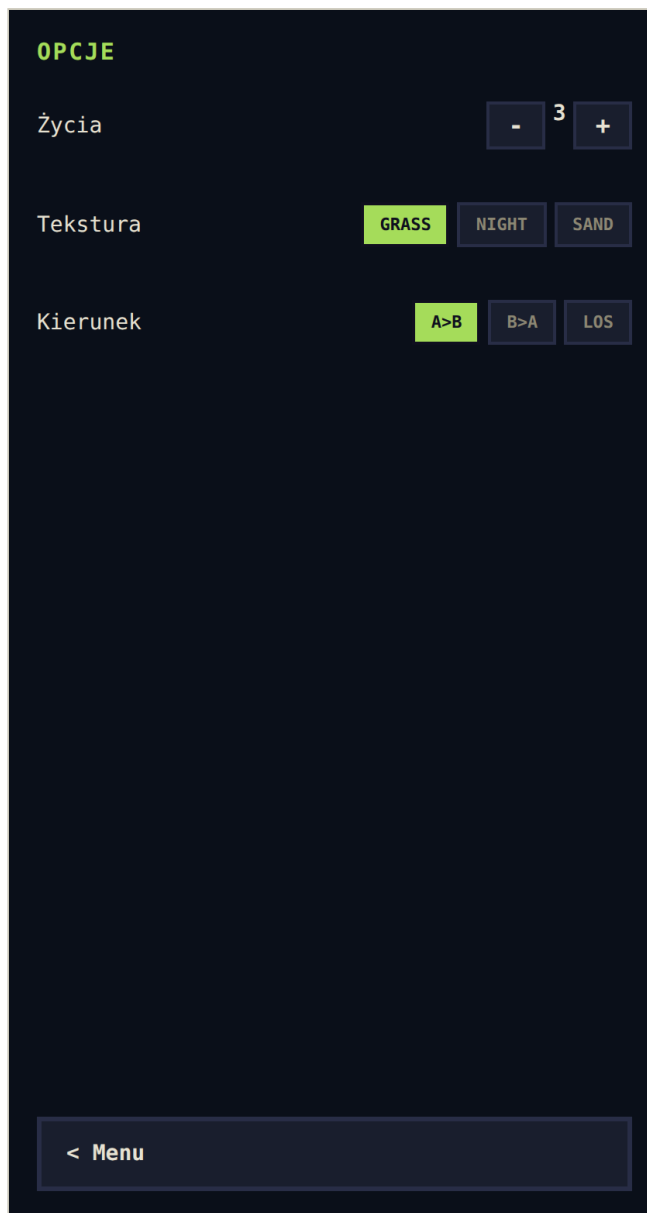
Gracz wybiera zestaw, a potem tryb — w tej wersji dostępny jest CLASSIC. Plansza wygląda tak: działko na środku, potworki nadchodzące z lewej i prawej krawędzi, nad każdym potworkiem hasło do przetłumaczenia. Pod planszą jest pole tekstowe i klawiatura.

Wpisywanie działa **po przedrostku**: gdy gracz zacznie pisać, podświetlają się wszystkie potworki, których odpowiedź zaczyna się od wpisanych liter. Po wpisaniu `cie` podświetli się `ciekawość`, i `cierpki` — dopiero dalsze litery je rozróżnią. Lufa działa obraca się w stronę potworka, do którego gracz aktualnie celuje.

Zatwierdzenie klawiszem Enter wystrzeliwuje pocisk, jeśli wpisany tekst dokładnie odpowiada któremuś z potworków. Jeżeli pasuje kilku, celem zostaje ten najbliższy działa. Zła odpowiedź nie odbiera życia — zeruje jedynie serię.



Rys. 3. Rozgrywka. U góry życia i zapas amunicji, u dołu punkty i poziom. Wpisane `pow` podświetla potworka z hasłem `serious`, bo jego odpowiedź brzmi `poważny`.



Rys. 4. Opcje. Liczba żyć, tekstura tła i kierunek tłumaczenia zmieniają zasady kolejnej rozgrywki.

Klawiatura ma litery A–Z, spację, cofanie, Enter i strzałki przesuwające kursor. Nie ma na niej polskich znaków — i nie są potrzebne, bo dopasowanie je pomija. Wpisane `zreczny` zalicza się tak samo jak `zręczny`. Rozdział 4.3 wyjaśnia, dlaczego tak jest.

3.3. Punkty, serie i poziomy

Za każdego zabitego potworka gracz dostaje $(10 + 2 \times \text{poziom}) \times \text{mnożnik}$ punktów. Mnożnik rośnie wraz z serią trafień bez pomyłki:

Trafienia z rzędu	Mnożnik	Punkty na poziomie 1	Punkty na poziomie 5
0–2	×1	12	20
3–5	×2	24	40
6–8	×3	36	60
9–11	×4	48	80
12 i więcej	×5	60	100

Serię zeruje zarówno błędna odpowiedź, jak i utrata życia. Nowy poziom przychodzi co **8 zabić**. Wraz z poziomem potworki idą szybciej i pojawiają się częściej: odstęp między nimi skraca się z 3,4 sekundy o 0,22 sekundy na poziom, aż do dolnej granicy jednej sekundy. Na planszy nigdy nie ma więcej niż siedmiu potworków naraz.

3.4. Pancerz i skrzynia z amunicją

Od **poziomu 3** część potworków nosi pancerz. Pierwszy pocisk go zbija, ale nie zabija — żeby zdjąć takiego potworka, trzeba wpisać jego tłumaczenie drugi raz. Prawdopodobieństwo pancerza rośnie o 12 punktów procentowych na poziom, do maksimum 60%.

Od **poziomu 2** co 19 sekund pojawia się skrzynia z amunicją, również opisana hasłem. Wpisanie jej tłumaczenia daje 3 mocniejsze naboje, które zabijają opancerzonego potworka za jednym razem. Gra dobiera je automatycznie, gdy są w zapasie. Skrzynia znika po 13 sekundach, a na planszy są najwyżej dwie naraz.

WSKAZÓWKA

Skrzynia to jedyna rzecz na planszy, która nie zabija gracza. Na wyższych poziomach opłaca się ją odpuścić, jeśli w tym samym czasie dwa potworki są blisko działka — pancerz kosztuje tylko dodatkowe wpisanie, a utracone życie jest nie do odzyskania.

3.5. Zestawy — własne i wbudowane

Ekran ZESTAWY pokazuje wszystkie zestawy: 102 wbudowane oraz te, które gracz utworzy sam. Edytor jest ułożony jak w Quizlecie — numerowane wiersze, po lewej hasło, po prawej tłumaczenie, z przyciskiem zamieniającym strony miejscami.

Jeżeli hasło ma kilka poprawnych tłumaczeń, rozdziela się je przecinkiem — każdy wariant będzie zaliczony osobno. Zapis **konsorcjum, syndykat** oznacza, że gra przyjmie oba słowa.

Usuwanie. Zestawy utworzone przez gracza mają przy sobie przycisk z koszem. Pierwsze tapnięcie uzbraja go — napis zmienia się na „Na pewno?” — a dopiero drugie kasuje zestaw. Zestawów wbudowanych nie da się usunąć; nie mają przycisku kosza.

3.6. Opcje

Ustawienie	Wartości	Domyślnie	Co zmienia
Życia	1–5	3	ile potworków może dojść do działka
Dźwięk	włączony / wyłączony	włączony	krótkie tony przy trafieniu, pomyłce i utracie życia
Tekstury	CLASSIC / NIGHT / DESERT	CLASSIC	zestaw kafli tła — trawa, noc albo piasek
Język interfejsu	polski / angielski	polski	napisy w aplikacji, nie treść zestawów
Kierunek tłumaczenia	hasło → tłumaczenie / tłumaczenie → hasło / losowo	hasło → tłumaczenie	którą stronę zestawu gracz wpisuje

Kierunek tłumaczenia jest najciekawszym z tych ustawień dydaktycznie. Tryb *tłumaczenie* → *hasło* jest wyraźnie trudniejszy, bo wymaga odtworzenia obcego słowa, a nie rozpoznania go — i to on właśnie ćwiczy czynne słownictwo.

3.7. Osiągnięcia i wyniki

Osiem odznak liczonych na bieżąco ze statystyk gracza:

Odznaka	Warunek	Odznaka	Warunek
Pierwsza krew	1 zabicie	Seria x3	seria 6 trafień
Rekrut	5 rozegranych gier	Seria x5	seria 12 trafień
Weteran	25 rozegranych gier	Pięćset	500 punktów w jednej grze
Strzelec wyborowy	100 zabić	Dwa tysiące	2000 punktów w jednej grze

Ekran WYNIKI pokazuje lokalny ranking, najlepsze przebiegi i najczęściej grany zestaw. Wszystko liczone jest z historii zapisanej na urządzeniu — nie ma rankingu globalnego, bo nie ma serwera.

4. Zawartość merytoryczna — słownictwo

4.1. Zakres i podział

Wbudowane słownictwo obejmuje **2084 hasła w 102 zestawach**, rozłożone na czternaście działów tematycznych zakresu matury podstawowej z języka angielskiego. Podstawą była lista 719 par przygotowana przez autora, rozbudowana następnie do pełnego zakresu tematycznego.

Dział	Zestawy	Dział	Zestawy
Człowiek	18	Zakupy	5
Miejsce zamieszkania	11	Podróże	6
Edukacja	8	Kultura	5
Praca	6	Sport	5
Życie prywatne	7	Zdrowie	5
Żywność	7	Nauka i technika	6
Przyroda	8	Życie społeczne	5

Nierówny podział jest zamierzony. Dział *Człowiek* obejmuje wygląd, charakter, uczucia, relacje i etapy życia, więc naturalnie rozpada się na osiemnaście zestawów. *Zakupy* mają zakres wyraźnie węższy i pięć zestawów w zupełności go pokrywa. Liczba zestawów odzwierciedla objętość materiału, a nie chęć zachowania symetrii w tabeli.

4.2. Zasada dwóch słów

Każde hasło i każde tłumaczenie mają **najwyżej dwa słowa**. Ograniczenie wynika wprost z mechaniki: hasło musi zmieścić się nad potworkiem idącym przez planszę, a odpowiedź trzeba zdążyć wpisać, zanim ten potworek dojdzie do działka. Całe zdanie nie spełnia żadnego z tych warunków.

Reguła nie jest zapisana wyłącznie w dokumentacji — pilnują jej dwa testy jednostkowe, osobno dla hasła i osobno dla każdego wariantu tłumaczenia. Przy każdym rozszerzaniu słownictwa wyłapywały naruszenia, których autor nie zauważył gołym okiem.

4.3. Dopasowanie odpowiedzi

Klawiatura w grze ma tylko litery A–Z. Gdyby dopasowanie wymagało polskich znaków, gracz nie mógłby wpisać poprawnej odpowiedzi na słowo *zręczny*. Zamiast dokładać do klawiatury dziewięć klawiszy — i zabierać miejsce na ekranie telefonu — porównanie odbywa się na tekście sprowadzonym do wspólnej postaci.

Normalizacja robi trzy rzeczy: zamienia wielkie litery na małe, składa znaki diakrytyczne do liter podstawowych (ą → a, ł → l, ż → z) i zbija wielokrotne odstępki do jednej spacji. Dopiero tak przygotowane teksty się porównuje.

```
"Zręczny"    → "zreczny"
"zreczny"    → "zreczny"    // wpisane przez gracza – zgodne
```

Tłumaczenia z kilkoma wariantami rozbijane są na przecinkach, średnikach i ukośnikach, a treść w nawiasach traktowana jest jako objaśnienie i pomijana. Zapis `zamek (budowla), pałac` daje trzy akceptowane odpowiedzi:

zamek budowla, zamek i pałac.

4.4. Wielkość zestawu

Zestawy mają od 20 do 25 haseł, przy czym większość ma dokładnie 20. Wartość nie jest przypadkowa: jedna rozgrywka pokazuje wtedy każde hasło kilka razy, więc powtórka odbywa się sama, bez osobnego trybu powtórek.

Pierwotny podział dawał zestawy od 8 do 89 haseł i był pod tym względem nieużyteczny — zestaw ośmiu haseł kończył się nudą po minucie, a w zestawie 89 haseł pojedyncze słowo wracało raz na kilkanaście minut. Przebudowa polegała na rozbiciu zestawów zbyt dużych, połączeniu zbyt małych i uzupełnieniu tych, którym po podziale brakowało kilku haseł do dwudziestu.

KONTROLA

Granice pilnuje test `every set is small enough to be cleared in one run`, który nie przepuszcza zestawu mniejszego niż 20 ani większego niż 25 haseł. Bez niego kolejne uzupełnienie słownictwa cicho rozjechałoby ten podział.

5. Architektura

5.1. Stos technologiczny

Element	Wersja	Rola
Kotlin	2.0.21	język całej aplikacji
Jetpack Compose	BOM 2024.10.01	interfejs i rysowanie planszy
Android Gradle Plugin	8.7.3	budowanie
Gradle	8.14.3	system budowania
kotlinx.serialization	1.7.3	zapis stanu do JSON
minSdk / targetSdk	24 / 35	Android 7.0 i nowsze
Kod bajtowy	Java 17	budowane pod JDK 21

Zależności zewnętrzne kończą się na tej liście. Nie ma biblioteki nawigacji, wstrzykiwania zależności ani warstwy sieciowej — aplikacja nie łączy się z niczym, a stos ekranów mieści się w kilkudziesięciu liniijkach.

5.2. Podział na moduły

Kod źródłowy to **16 plików Kotlina, 6455 linii**, ułożonych w cztery pakiety:

Plik	Linii	Odpowiedzialność
MainActivity.kt	17	punkt wejścia — jedna aktywność, cała reszta w Compose
model/Models.kt	104	typy danych: zestaw, hasło, opcje, profil, wynik, osiągnięcia
game/GameEngine.kt	499	cała symulacja rozgrywki
game/Matching.kt	83	normalizacja i porównywanie odpowiedzi
data/Repository.kt	96	odczyt i zapis stanu
data/BuiltInSets.kt	3224	wbudowane słownictwo (generowane)
ui/PlayScreen.kt	503	rysowanie planszy i pętla klatek
ui/Screens.kt	732	dziewięć ekranów poza rozgrywką
ui/Sprites.kt	280	dobór sprite'ów, animacje, kafle tła
ui/PixelFont.kt	233	rysowanie tekstu fontem bitmapowym
ui/Keyboard.kt	131	klawiatura ekranowa
ui/App.kt	148	stos ekranów i stan aplikacji
ui/Common.kt , Theme.kt , Strings.kt , Sound.kt	405	elementy wspólne, paleta, napisy PL/EN, dźwięk

DECYZJA PROJEKTOWA

`BuiltInSets.kt` ma 102 osobne funkcje prywatne, po jednej na zestaw, zamiast jednej wielkiej listy. Powód jest techniczny: metoda w kodzie bajtowym Javy nie może przekroczyć 64 kB, a 2084 hasła zapisane w jednym ciele funkcji ten limit przekraczają. Plik jest generowany skryptem, więc taki podział nic nie kosztuje w utrzymaniu.

5.3. Silnik gry

Kluczowa cecha silnika: **nie zna żadnego typu z Androida ani z Compose**. Operuje na liczbach i własnych klasach danych, przez co da się go uruchomić w zwykłym teście na JVM, bez emulatora. Plansza to kwadrat jednostkowy — działko stoi w punkcie (0,5; 0,5), a potworki wchodzą zza krawędzi. Przeliczeniem tego na piksele zajmuje się dopiero warstwa rysująca.

Stała	Wartość	Znaczenie
<code>CANNON_RADIUS</code>	0,06	promień działka — dotknięcie odbiera życie
<code>BASE_SPEED</code>	0,030	prędkość potworka na poziomie 1 (część planszy na sekundę)
<code>SPEED_PER_LEVEL</code>	0,0055	przyrost prędkości na poziom
<code>PROJECTILE_SPEED</code>	1,25	prędkość pocisku
<code>BASE_SPAWN_INTERVAL</code>	3,4 s	odstęp między potworkami na starcie
<code>MIN_SPAWN_INTERVAL</code>	1,0 s	najkrótszy możliwy odstęp
<code>KILLS_PER_LEVEL</code>	8	zabić na nowy poziom
<code>MAX_MONSTERS</code>	7	limit potworków na planszy
<code>CRATE_INTERVAL</code> / <code>CRATE_TTL</code>	19 s / 13 s	jak często pojawia się skrzynia i jak długo trwa
<code>WALK_FRAME_TIME</code>	0,2 s	tempo animacji chodu

Silnik posuwa się metodą `tick(dt)` wywoływaną raz na klatkę. W jednym kroku przesuwa pociski, rozstrzyga trafienia, przesuwa potworki, sprawdza dotknięcia działka, usuwa przeterminowane skrzynie i decyduje o kolejnym pojawieniu.

5.4. Zapis danych

Cały stan — profil, opcje, zestawy i historia wyników — mieści się w jednym dokumencie JSON w prywatnym katalogu aplikacji (`verbume.json`). Zapis jest atomowy w tym sensie, że zawsze podmienia całość, więc nie da się doprowadzić do stanu częściowo zaktualizowanego.

Odczyt jest odporny na uszkodzony albo starszy plik: gdy dekodowanie się nie powiedzie, aplikacja startuje od stanu początkowego zamiast się wywracać. Włączona jest też opcja `ignoreUnknownKeys`, dzięki której plik zapisany przez nowszą wersję nie blokuje wersji starszej.

5.5. Grafika i font bitmapowy

Oprawa to 97 plików PNG w rozdzielczości natywnej, wszystkie w katalogu `drawable-nodpi`. Nazwa katalogu jest tu istotna — mówi Androidowi, żeby przy wczytywaniu **nie przeskalowywał** obrazków pod gęstość ekranu. Bez tego

pixel-art rozmywa się już na etapie ładowania, zanim cokolwiek narysujemy.

Każde rysowanie używa dodatkowo trybu `FilterQuality.None`, czyli powiększania metodą najbliższego sąsiada. Kafle tła układane są krokiem całkowitoliczbowym, bo przy kroku ułamkowym zaokrąglenia zostawiają cienkie szpary między kafłami.

Napisy w interfejsie nie korzystają z kroju systemowego. Rysuje je własny renderer, który wycina glify 5×7 z arkusza `font_8x8` i składa z nich tekst, dokładając kontur przez narysowanie tego samego napisu przesuniętego o piksel. Polskie znaki są w arkuszu jako osobne glify — dzięki temu wyglądają tak samo jak reszta napisów, zamiast wypadać z konwencji.

5.6. Demo przeglądarkowe

Demo to **jeden samodzielny plik HTML** zawierający wszystko: logikę gry przepisaną na JavaScript, 97 sprite'ów wklejonych jako adresy `data:` i pełne słownictwo. Nie pobiera niczego z sieci, więc działa również otwarte z dysku.

Plik powstaje ze skryptu budującego, który skleja szablon HTML, zakodowaną grafikę, wyeksportowane zestawy i wspólne ciało JavaScriptu. Ta sama procedura tworzy demo Aleimy — różni się plikiem z ciałem, słownictwem i etykietami.

ZAUWAŻONY BŁĄD

Oba demo stoją pod jedną domeną `kubakozub.github.io`, a pamięć przeglądarki jest wspólna dla całej domeny, nie dla pojedynczego adresu. Demo Aleimy używało początkowo tych samych kluczy `verbume_*`, przez co pokazywałyoby zestawy z Verbume i dzieliło z nią rekord punktowy. Aleima ma dziś własne klucze `aleima_*`.

6. Testy

6.1. Co jest sprawdzane

Projekt ma **44 testy jednostkowe** w czterech klasach. Wszystkie przechodzą.

Klasa	Testów	Zakres
GameEngineTest	21	ruch potworków, trafienia, pancerz, skrzynie, poziomy, mnożniki, utrata życia, koniec gry
BuiltInSetsTest	10	reguły słownictwa: długość haseł, brak duplikatów, wielkość zestawów, komplet działów
MatchingTest	7	normalizacja, diakrytyki, warianty po przecinku, dopasowanie po przedrostku
PersistenceTest	6	zapis i odczyt stanu, odporność na uszkodzony plik

Testy słownictwa zasługują na osobną uwagę, bo nie sprawdzają kodu, tylko treść. Sprawdzają między innymi, że każde hasło ma najwyżej dwa słowa, że w jednym zestawie nie powtarza się to samo hasło, że każde tłumaczenie da się wpisać na klawiaturze A–Z i że obecne są wszystkie czternaście działów. W praktyce wyłapywały naruszenia przy każdym większym rozszerzeniu listy — kilkanaście przy pierwszym uzupełnieniu, kilkadziesiąt przy ostatnim.

6.2. Testowalność silnika

Silnik daje się testować, ponieważ nie zależy od Androida. Ma też jedno celowe ułatwienie: konfiguracja rozgrywki przyjmuje przełącznik `autoSpawn`, którym można wyłączyć samoczynne pojawianie się potworków.

DLACZEGO TAK

Bez tego przełącznika testy bywały niestabilne. Test ustawiał potworka, sprawdzał trafienie — a w międzyczasie silnik dorzucał kolejnego i asercja o liczbie potworków przestawała się zgadzać. Wyłączenie automatycznego pojawiania w testach jednostkowych rozwiązuje to u źródła. Samo automatyczne pojawianie nie zostało przez to bez pokrycia: sprawdza je osobny test limitu siedmiu potworków, który celowo działa z włączonym `autoSpawn`.

7. Decyzje projektowe

Zestawienie rozstrzygnięć, które najmocniej ukształtowały tę wersję, wraz z powodami.

Dopasowanie bez polskich znaków

Klawiatura ekranowa ma tylko A–Z, a porównanie odpowiedzi pomija diakrytyki. Alternatywą było dołożenie dziewięciu klawiszy, co na ekranie telefonu zabiera miejsce potrzebne planszy. Koszt tej decyzji jest realny — gra nie nauczy poprawnej pisowni z ogonkami — ale ćwiczy słownictwo, a nie ortografię, i to jest jej zadanie.

Silnik bez zależności od Androida

Cała logika gry operuje na zwykłych liczbach i klasach danych. Dzięki temu 44 testy działają w kilkanaście sekund na zwykłej maszynie wirtualnej Javy. Gdyby silnik dotykał typów Androida, te same testy wymagałyby emulatora — a środowisko, w którym projekt powstawał, nie miało sprzętowej wirtualizacji i emulatora nie dałoby się uruchomić.

Brak biblioteki nawigacji

Ekran trzymane są na zwykłym stosie klas zapieczętowanych. Aplikacja ma dziewięć ekranów i żadnych głębokich odnośników, więc pełna biblioteka nawigacji dokładałaby zależność i konfigurację bez korzyści.

Brak kont i serwera

Dokument projektowy przewidywał logowanie, współdzielenie zestawów i ranking globalny. Wszystko to wymaga serwera, jego utrzymania i obchodzenia się z danymi użytkowników. Ta wersja świadomie zostaje przy grze anonimowej i zapisie lokalnym — działa bez internetu i nie zbiera o gracz niczego.

Dźwięk z tonów systemowych

Zamiast plików audio gra używa krótkich tonów generowanych przez system. Odróżniają trafienie, zabicie pancerza, pomyłkę, zebranie skrzyni i utratę życia. To zauważalnie uboższe niż prawdziwe efekty, ale nie wymaga dołączania nagrań, których licencję trzeba by rozstrzygnąć.

Dwustopniowe usuwanie zestawu

Kosz przy własnym zestawie kasował go od razu po jednym tapnięciu. Na ekranie dotykowym, przy przycisku sąsiadującym z „Edytuj”, jest to zaproszenie do pomyłki nie do cofnięcia — zestaw wpisany ręcznie znika bezpowrotnie. Pierwsze tapnięcie uzbraja więc przycisk, a dopiero drugie kasuje.

8. Realizacja dokumentu projektowego

Zestawienie tego, co z pierwotnego dokumentu *Verbume* — *Verbum* + *Game* zostało zbudowane, a co nie.

Zrealizowane

Założenie	Stan
Działko na środku z ruchomą lufą	lufa celuje w potworka, do którego gracz aktualnie pisze
Potworki wychodzące z boków planszy	tak, z obu krawędzi, na losowej wysokości
Podświetlanie po przedrostku	tak, z testem odtwarzającym przykład z dokumentu
Ograniczona klawiatura z kursorem	A–Z, spacja, cofanie, Enter, strzałki
Życia, konfigurowalne	1–5, domyślnie 3
Licznik amunicji, najslabsza nieskończona	wyświetlany jako ∞ / N
Punkty i poziom trudności w rogach planszy	tak
Skrzynie z amunicją na wyższych poziomach	od poziomu 2
Pancerz wymagający dwóch trafień	od poziomu 3
Mocniejsze naboje zabijające opancerzonego	dobierane automatycznie z zapasu
Mnożniki x2–x5 za serię	progi 3, 6, 9 i 12 trafień
Rosnąca trudność	nowy poziom co 8 zabić
Gra anonimowa	jedyna dostępna ścieżka
Menu: PLAY / SETS / OPTIONS / ACHIEVEMENTS / SCORES / EXIT	komplet
Edytor zestawów w układzie Quizleta	numerowane wiersze, zamiana stron
Opcje: dźwięk, życia, tekstury, język, kierunek	komplet
Osiągnięcia i ranking	8 odznak, ranking lokalny

Niezrealizowane

Założenie	Powód
Tryby TRUE/FALSE, MULTIPLE CHOICE, TIME ATTACK	zaprojektowane w dokumencie, nie zbudowane w tej wersji — menu wyboru trybu pokazuje je jako „wkrótce”
Konta i logowanie	wymaga serwera
Współdzielenie zestawów między graczami	wymaga serwera
Ranking globalny	wymaga serwera

Rozróżnienie jest w aplikacji widoczne, a nie ukryte: ekran wyboru trybu pokazuje pozostałe tryby jako zablokowane wraz z informacją, że są opisane w dokumencie, ale nie ma ich w tej wersji.

9. Ograniczenia i kierunki rozwoju

Znane ograniczenia

- **Brak testów interfejsu.** Pokryty testami jest silnik, dopasowanie odpowiedzi, zapis i treść słownictwa. Warstwa rysująca sprawdzana była wyłącznie ręcznie oraz przez oglądanie zrzutów ekranu — właśnie tak wyszły na jaw źle umieszczony pancerz i szpary między kaflami tła.
- **Wydania podpisane kluczem debugowym.** Wystarcza to do instalacji z pliku, ale nie do publikacji w sklepie.
- **Skrypty generujące słownictwo są poza repozytorium.** Plik `BuiltInSets.kt` jest wynikiem ich działania, ale one same nie są wersjonowane — odtworzenie procesu z samego repozytorium nie jest dziś możliwe.
- **Gra nie uczy pisowni z ogonkami,** co jest bezpośrednim skutkiem decyzji o klawiaturze A–Z.
- **Brak trybu powtórek opartego na błędach.** Gra nie zapamiętuje, na których hasłach gracz się myli, więc nie potrafi ich później podsuwać częściej.

Naturalne następne kroki

1. **Powtórki oparte na pomyłkach.** Zapisywanie, które hasła sprawiają kłopot, i częstsze ich pokazywanie. To pojedyncza zmiana w doborze słowa przy pojawieniu potworka, a dydaktycznie najmocniejsze usprawnienie z całej listy.
2. **Pozostałe tryby z dokumentu.** TRUE/FALSE i MULTIPLE CHOICE nie wymagają nowej mechaniki — tylko innego sposobu zadawania pytania na tym samym silniku.
3. **Import z Quizleta w aplikacji.** Demo przeglądarkowe już przyjmuje wklejony eksport; aplikacja na Androida wymaga wpisywania haseł ręcznie.
4. **Włączenie skryptów generujących do repozytorium,** żeby słownictwo dało się odtworzyć i rozszerzyć bez dostępu do maszyny, na której powstawało.
5. **Podpisanie wydań własnym kluczem,** jeśli aplikacja miałaby kiedyś trafić do sklepu.

A. Załącznik: struktura repozytorium

```
Verbume/ (gałąź master)
├── app/
│   ├── build.gradle.kts          konfiguracja modułu, wersja 0.5.0
│   └── src/
│       ├── main/
│       │   ├── java/com/verbume/app/
│       │   │   ├── MainActivity.kt
│       │   │   ├── model/Models.kt
│       │   │   ├── game/GameEngine.kt, Matching.kt
│       │   │   ├── data/Repository.kt, BuiltInSets.kt
│       │   │   └── ui/ (11 plików: ekrany, grafika, font, klawiatura)
│       │   ├── res/drawable-nodpi/ 97 plików PNG w skali natywnej
│       │   └── assets/font-layout.md układ glifów w arkuszu fontu
│       └── test/java/com/verbume/app/
│           ├── EngineTest.kt      GameEngineTest + MatchingTest
│           ├── BuiltInSetsTest.kt reguły słownictwa
│           └── PersistenceTest.kt zapis i odczyt
├── docs/
│   ├── index.html                demo przeglądarkowe (GitHub Pages)
│   ├── pobierz.html              strona pobierania APK
│   └── .nojekyll
├── build.gradle.kts
├── settings.gradle.kts
├── gradle/wrapper/
└── README.md
```

Plik `local.properties` ze ścieżką do Android SDK jest celowo pominięty w repozytorium — jest zależny od maszyny i każdy tworzy go u siebie. Katalog `docs/` jest źródłem dla GitHub Pages: w ustawieniach repozytorium trzeba wskazać gałąź **master** i folder `/docs`.

Publikacja

Co	Gdzie
Kod źródłowy	<code>github.com/kubakozub/Verbume</code> , gałąź <code>master</code>
Demo przeglądarkowe	<code>kubakozub.github.io/Verbume</code>
Plik APK	budowany lokalnie; nie jest wersjonowany w repozytorium

Plik APK jest świadomie wykluczony z historii gita. Jednorazowe wrzucenie go do repozytorium rozdmuchało katalog `.git` z kilkuset kilobajtów do jedenastu megabajtów, a ponieważ git przechowuje każdą wersję każdego pliku, kolejne wydania powiększałyby go w nieskończoność.